

APLICACIONES WEB • 5.º SEMESTRE

Unidad II

Programación del lado del servidor

PERL • PHP • Java/Spring Boot • ASP.NET Core • Seguridad (OWASP)

Semanas 5 a 9

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Universidad Técnica Estatal de Quevedo • gguerrero@uteq.edu.ec

Hoja de ruta: cinco semanas, un mismo eje

SEMANA 5	SEMANA 6	SEMANA 7	SEMANA 8	SEMANA 9
PERL y PHP	Java / Spring Boot	ASP.NET Core	Tutoría de refuerzo	Seguridad + EP1
Modelo request-response. CGI historico y PHP 8.3: el caballo de batalla de la web tradicional.	Servlets y JSP como cimiento; Spring Boot 3 + JPA como estandar moderno de la industria.	La plataforma .NET multiplataforma. Web API CRUD con C# 12 y Entity Framework Core.	Consolidacion del primer corte: repaso de errores frecuentes y preparacion de la EP1.	OWASP Top 10:2021, defensas tecnicas basicas y Evaluacion Parcial Primer Corte.

Eje transversal: el ciclo petición → procesamiento → respuesta. La seguridad (semana 9) NO se añade al final: se construye desde la semana 5.

Resultados de aprendizaje de la unidad

Al finalizar la Unidad II, el estudiante será capaz de:

- Explicar el modelo request-response de HTTP y la arquitectura navegador → servidor web → intérprete.
- Desarrollar aplicaciones dinámicas del lado del servidor en PHP 8.3 con procesamiento seguro de formularios.
- Construir APIs REST y un CRUD funcional con Spring Boot 3 + JPA y con ASP.NET Core 8 + EF Core.
- Identificar las diez vulnerabilidades del OWASP Top 10:2021 y aplicar defensas básicas.
- Rendir con éxito la Evaluación Parcial Primer Corte (semanas 1–8).

¿Por qué el servidor importa?

El cliente es manipulable

Cualquier validación o regla de negocio que viva solo en el navegador puede ser saltada con las DevTools (F12). El servidor es la única frontera de confianza.

El servidor es donde vive el valor

Base de datos, autenticación, pagos, lógica de negocio: todo lo crítico se ejecuta aquí, fuera del alcance del usuario.

Un mismo patrón, varios lenguajes

PHP, Java y C# resuelven el mismo problema con sintaxis distinta. Aprender el patrón importa más que el lenguaje.

SEMANA 5

Introducción a la programación del lado del servidor: PERL y PHP

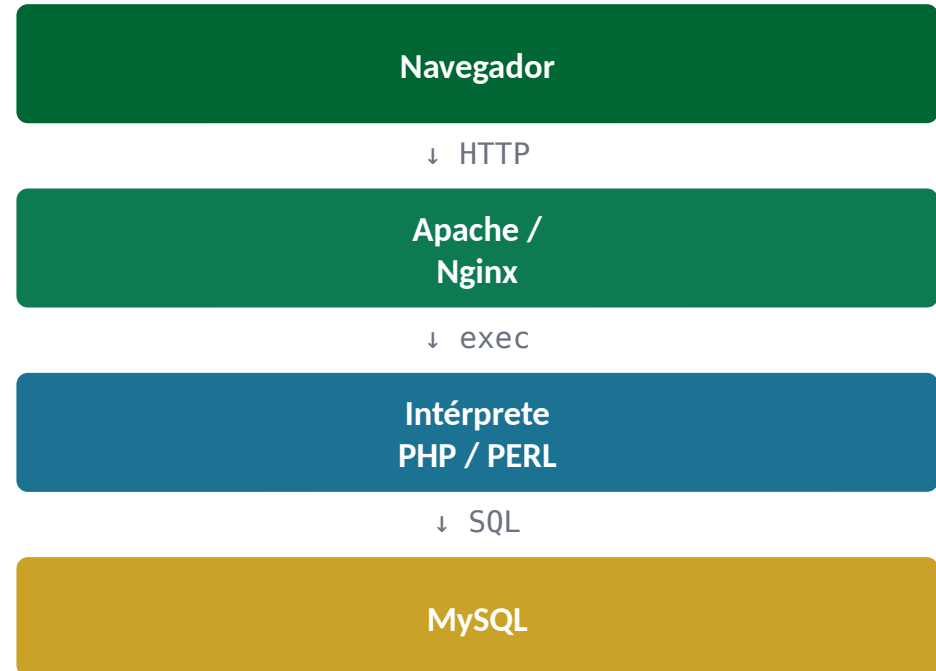
«PHP creció orgánicamente para resolver problemas reales. PERL fue diseñado para ser práctico. Esa pragmática es lo que hizo a ambos sobrevivientes.» — Adaptado de Rasmus Lerdorf y Larry Wall

El modelo request-response de HTTP

La conversación cliente ↔ servidor

- El navegador (cliente) envía una petición HTTP (request) al servidor.
- El servidor de aplicaciones ejecuta código (PHP, Java, .NET, Python) y genera dinámicamente una respuesta.
- La respuesta (response) depende de la petición, los datos de sesión, la base de datos, etc.
- HTTP es sin estado (stateless): cada petición nace y muere sin memoria de las anteriores.
- Por eso necesitaremos cookies y sesiones más adelante (semana 10).

Flujo de una petición PHP/PERL



PERL para CGI: el pionero de la web dinámica

L Larry Wall y la web de los 90

- Larry Wall, lingüista y programador, creó PERL en 1987.
- En 1993 PERL+CGI fue LA combinación que hizo dinámica a la web: formularios, correos, páginas generadas.
- Amazon, Slashdot y muchos e-commerce pioneros nacieron en PERL.
- Su lema TIMTOWTDI («hay más de una forma de hacerlo») lo hizo expresivo pero difícil de mantener.
- En 2026 sobrevive en sysadmin, integración y análisis de texto; en la web quedó relegado a sistemas legados.

```
#!/usr/bin/perl
use strict;           # disciplina obligatoria
use warnings;         # avisa de errores comunes
use CGI;              # modulo para web (Common
                      # Gateway Interface)

my $q = CGI->new;
my $nombre = $q->param('nombre') || 'Mundo';

print $q->header('text/html'); # cabecera HTTP
print "<h1>Hola, $nombre desde PERL/CGI</h1>";
```

Hola mundo en PERL/CGI – recibe ?nombre por GET

PHP: el lenguaje dominante de la web tradicional

Tabla evolutiva de PHP

Año	Versión	Aporte clave
1995	PHP 1.0	Rasmus Lerdorf publica «Personal Home Page Tools».
2004	PHP 5	Modelo de objetos serio; nace la web PHP moderna.
2015	PHP 7	Salto de rendimiento (≈2×) y tipos escalares.
2020	PHP 8	JIT, named arguments, match, atributos.
2023	PHP 8.3	Tipado más estricto, mejoras de rendimiento.

`$_GET` vs `$_POST`: GET para leer/consultar (idempotente, visible en URL); POST para enviar/modificar datos.

Procesamiento de formularios

```
<?php
// $_GET -> datos en la URL (?q=...)
// $_POST -> datos del cuerpo (formulario)
$nombre = $_POST['nombre'] ?? '';

// NUNCA confiar en la entrada: validar
$nombre = trim($nombre);
if ($nombre === '')
    die('El nombre es obligatorio');

// Escapar SIEMPRE la salida (anti-XSS)
echo 'Hola, ' .
    htmlspecialchars($nombre);
?>
```

Buenas prácticas profesionales en PHP

1

`declare(strict_types=1)`

Tipado estricto en todo archivo.

2

PDO + prepared statements

Nunca concatenar SQL con datos del usuario.

3

`htmlspecialchars` en la salida

Defensa anti-XSS por defecto.

4

`password_hash()` (BCrypt)

Jamás MD5/SHA1 para contraseñas.

5

Composer + PSR-4

Autoload y estándar de organización.

6

UTF-8 de extremo a extremo

HTML, BD, conexión y cabecera HTTP.

SEMANA 6

Java para aplicaciones web: JSP, Servlets y Spring Boot

Del Servlet puro al framework opinado: cómo Java pasó de ser verboso a ser el estándar empresarial productivo de 2026.

Cinco generaciones de Java en la web

Año	Versión	Aporte
2014	Java 8	Lambdas, streams, Optional.
2014	Spring Boot 1.0	Convención sobre configuración.
2018	Java 11 LTS	Versión LTS estable.
2021	Java 17 LTS	Sealed classes, records, pattern matching.
2022	Spring Boot 3.0	Jakarta EE 9 (paquetes jakarta.*).
2023	Java 21 LTS	Virtual threads, sequenced collections.
2024	Spring Boot 3.4	Compatible con Java 21+, observabilidad.
2025	Java 25 LTS	Patrones de cadena en switch.

Lo que hay que recordar

- Usar siempre una versión LTS (soporte a largo plazo): Java 17 o 21.
- Spring Boot 3 requiere Java 17+ y migró de javax.* a jakarta.* (cambio que rompe código antiguo).
- El PFC del curso usa Spring Boot 3.4 + Java 21.
- Virtual threads (Java 21) permiten manejar miles de peticiones concurrentes baratas.

El motor histórico y el estándar moderno

Servlet puro (el cimiento)

```
@WebServlet("/hola")
public class HolaServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req,
                          HttpServletResponse resp){
        resp.getWriter().write("Hola Mundo"); } }
```

Spring Boot (lo mismo, productivo)

```
@RestController
public class HolaController {
    @GetMapping("/hola")
    public String hola() { return "Hola Mundo"; }
}
```

¿Qué aporta Spring Boot?

- **Opinionado:** trae decisiones por defecto sensatas (convención sobre configuración).
- **Auto-configuration:** detecta las dependencias y se configura solo.
- **Servidor Tomcat embebido:** no se instala aparte; la app se ejecuta con un clic.
- **Starters:** una dependencia («spring-boot-starter-web») trae todo lo necesario.
- **@RestController + @GetMapping:** un método Java se convierte en endpoint REST que devuelve JSON.
- **Resultado:** menos código repetitivo (boilerplate), foco en la lógica de negocio.

Spring Data JPA: del objeto a la tabla sin escribir SQL

Entidad + Repositorio

```
@Entity                                // mapea a tabla
@Table(name = "productos")
public class Producto {
    @Id @GeneratedValue(strategy = IDENTITY)
    private Long id;                    // clave primaria
    private String nombre;
    private BigDecimal precio;
}

// JPA genera el SQL automaticamente:
public interface ProductoRepository
    extends JpaRepository<Producto, Long> { }
// findAll(), findById(), save(), delete()...
```

Las capas del backend

@RestController

Recibe la petición HTTP

@Service

Reglas de negocio

@Repository (JPA)

Acceso a la base de datos

Base de datos

PostgreSQL / H2

Doce reglas profesionales en Spring Boot

1. Controllers thin: la lógica va en servicios.
2. Capas: controller → service → repository.
3. DTOs separados de las entidades JPA.
4. @Valid en TODA entrada de los controllers.
5. Manejo global de excepciones (@ControllerAdvice).
6. Patrón PRG (Post-Redirect-Get).
7. Lombok para reducir boilerplate.
8. Slf4j + Logback con niveles correctos.
9. Tests con @WebMvcTest y @DataJpaTest.
10. Profiles separados: dev, test, prod.
11. Configuración externa: application-{profile}.yml.
12. Health checks con Spring Boot Actuator.

SEMANA 7

ASP.NET Core: la plataforma .NET para la web moderna

«La plataforma .NET pasó de propietaria y Windows-only a open-source y multiplataforma. Es el mayor giro estratégico de Microsoft en 20 años.» — Adaptado de Scott Hanselman

De ASP clásico a ASP.NET Core multiplataforma

1996	ASP clásico	Active Server Pages. Propietario, Windows-only, VBScript.
2002	ASP.NET WebForms	.NET Framework, modelo de eventos tipo Windows Forms.
2009	ASP.NET MVC	Inspirado en Ruby on Rails: separación de capas.
2016	ASP.NET Core 1.0	Open source, multiplataforma (Linux/Mac/Win), modular.
2023	ASP.NET Core 8	Versión estable actual; rendimiento comparable a Java/Go.
2024-26	Core 9 y 10	9 ya disponible; 10 en preview. Hoja de ruta abierta.

2016 es el punto de inflexión: el giro a open source y multiplataforma.

La CLI dotnet y la estructura del proyecto

Comandos esenciales de la CLI dotnet

```
dotnet --version          # version instalada
dotnet new webapi -n MiApi  # crear Web API
dotnet new mvc -n MiMvc    # crear app MVC
dotnet restore            # baja dependencias
dotnet build              # compila
dotnet run                # ejecuta
dotnet watch run          # con hot reload
dotnet test               # corre los tests
dotnet publish -c Release # para produccion
dotnet ef migrations add Inicial # BD
dotnet ef database update  # aplica
```

Estructura típica del proyecto

```
MiApp/
├─ Controllers/    # API o MVC
├─ Models/         # entidades y DTOs
├─ Views/          # Razor (solo MVC)
├─ wwwroot/        # recursos estaticos
├─ Program.cs      # punto de entrada
├─ appsettings.json # configuracion
└─ MiApp.csproj    # def. del proyecto
```

NuGet es el gestor de paquetes de .NET (equivalente a npm o Maven).

Web API CRUD con EF Core y Swagger

Controlador REST en C#

```
[ApiController]
[Route("api/[controller]")]
public class ProductosController
    : ControllerBase {
    private readonly ApplicationDbContext _ctx;

    [HttpGet] // GET /api/productos
    public async Task<...> Listar()
        => await _ctx.Productos.ToListAsync();

    [HttpPost] // POST -> crea, 201 Created
    public async Task<...> Crear(Producto p){
        _ctx.Productos.Add(p);
        await _ctx.SaveChangesAsync(); ... } }
```

Tres estilos en ASP.NET Core

Web API

Solo JSON, sin vistas. Ideal para SPA + REST.

MVC

Controllers + vistas Razor + models.

Razor Pages

Cada página tiene su PageModel.

Para el PFC: Web API + frontend Angular separado (patrón profesional moderno).

Buenas prácticas y comparativa Spring Boot ↔ ASP.NET

Buenas prácticas ASP.NET Core

- [ApiController] para validación automática del modelo.
- DTOs y validación con Data Annotations ([Required], [Range]).
- Inyección de dependencias nativa (constructor injection).
- async/await en TODO acceso a base de datos.
- Códigos HTTP semánticos (200, 201, 204, 404).
- Swagger/OpenAPI habilitado en desarrollo.
- Cadenas de conexión y secretos fuera del código (appsettings, user-secrets).

Mismo problema, dos plataformas

Concepto	Spring Boot	ASP.NET Core
Lenguaje	Java	C#
Controlador	@RestController	[ApiController]
GET	@GetMapping	[HttpGet]
ORM	JPA / Hibernate	EF Core
Servidor	Tomcat embebido	Kestrel
Paquetes	Maven / Gradle	NuGet

SEMANA 8

Tutoría de refuerzo: consolidación y resolución de blockers

Sin tema nuevo. Esta semana se consolida lo aprendido en las semanas 1-7 y se prepara la Evaluación Parcial Primer Corte.

Los tres temas que más cuestan

GET vs POST

GET: leer/consultar, idempotente, parámetros visibles en la URL, cacheable. POST: enviar/modificar, datos en el cuerpo, no debe repetirse al recargar. Usar GET en un login es un error de seguridad.

Validación cliente vs servidor

El cliente (HTML5/JS) valida para mejorar la experiencia (UX), pero NUNCA es la única defensa: con las DevTools se salta. El servidor (PHP/Java/.NET) DEBE revalidar TODA entrada porque el cliente es manipulable.

Codificación UTF-8

Los caracteres raros en vez de tildes vienen de un fallo de codificación. UTF-8 debe coincidir en CUATRO lugares: la etiqueta `<meta charset>` del HTML, los archivos guardados sin BOM, la base de datos (utf8mb4) y la cabecera HTTP Content-Type.

SEMANA 9

Seguridad en aplicaciones web y Evaluación Parcial Primer Corte

«El software seguro no es un producto: es un proceso continuo. Cada commit es una oportunidad de introducir o eliminar una vulnerabilidad.» — Adaptado de Bruce Schneier

OWASP Top 10:2021 — las diez vulnerabilidades críticas

ID	Categoría	Defensa esencial
A01	Broken Access Control	Autorización en cada endpoint del servidor, no en el cliente.
A02	Cryptographic Failures	Cifrar datos sensibles; algoritmos fuertes; HTTPS.
A03	Injection (SQL, etc.)	Prepared statements / consultas parametrizadas.
A04	Insecure Design	Threat modeling desde el diseño.
A05	Security Misconfiguration	Endurecer config.; quitar valores por defecto.
A06	Vulnerable Components	Dependencias actualizadas (Dependabot, Snyk).
A07	Auth. Failures	MFA, rate limiting, gestión de sesiones segura.
A08	Integrity Failures	Verificar firmas; cuidar la cadena de suministro.
A09	Logging Failures	Registrar y monitorizar eventos de seguridad.
A10	SSRF	Validar y restringir las URLs que el servidor solicita.

Cinco defensas técnicas que debes aplicar siempre

- 1 Anti-SQL Injection** Prepared statements (PDO, JDBC, EF Core). El dato del usuario nunca se concatena en la consulta.
- 2 Anti-XSS** Escapar la salida: htmlspecialchars en PHP, escape automático en Thymeleaf/Razor.
- 3 Cookies de sesión seguras** Flags HttpOnly (no accesible por JS), Secure (solo HTTPS) y SameSite (anti-CSRF).
- 4 Hash de contraseñas** BCrypt o Argon2 con factor de coste. JAMÁS guardar contraseñas en texto ni con MD5/SHA1.
- 5 Cabeceras de seguridad HTTP** HSTS (fuerza HTTPS), CSP (restringe orígenes), X-Frame-Options (anti-clickjacking).

Reglas de seguridad y estructura de la EP1

Reglas de seguridad imprescindibles

- HTTPS obligatorio (TLS 1.2 mínimo, 1.3 ideal).
- Nunca confiar en datos del cliente: revalidar siempre.
- Prepared statements en TODA consulta SQL.
- Escapar TODA salida (htmlspecialchars o equivalente).
- BCrypt/Argon2 para contraseñas; jamás MD5/SHA1.
- Cookies con HttpOnly, Secure y SameSite.
- Sin secretos en el repositorio (.env, vault).
- Mantener dependencias actualizadas (Dependabot, Snyk).

Evaluación Parcial Primer Corte (EP1)

Parte	Tipo	Pts
A	Selección múltiple + V/F	20
B	Respuesta corta (3 líneas)	20
C	Análisis de código (errores)	30
D	Ejercicio integrador	30

Temas garantizados:

HTML5/WCAG • CSS3/Flexbox/Grid • JS/DOM/fetch • PHP 8 con PDO • Spring Boot 3 + JPA • ASP.NET Core 8 • OWASP (A03 y A07).

Glosario de siglas y acrónimos (1/2)

HTTP — HyperText Transfer Protocol: protocolo de la web.

HTTPS — HTTP Secure: HTTP cifrado con TLS.

TLS — Transport Layer Security: cifra el canal.

CGI — Common Gateway Interface: ejecutar programas en el servidor.

PHP — PHP: Hypertext Preprocessor (acrónimo recursivo).

PERL — Practical Extraction and Report Language.

PDO — PHP Data Objects: capa de acceso a datos.

SQL — Structured Query Language: consulta a bases de datos.

CRUD — Create, Read, Update, Delete.

REST — Representational State Transfer: estilo de API.

JSON — JavaScript Object Notation: formato de datos.

API — Application Programming Interface.

JSP — JavaServer Pages: plantillas con Java embebido.

JPA — Jakarta Persistence API: ORM estándar de Java.

ORM — Object-Relational Mapping: objetos ↔ tablas.

DTO — Data Transfer Object: objeto de transferencia.

DI — Dependency Injection: inyección de dependencias.

LTS — Long-Term Support: soporte a largo plazo.

Glosario de siglas y acrónimos (2/2)

EF Core — Entity Framework Core: ORM de .NET.

CLI — Command-Line Interface: línea de comandos.

NuGet — Gestor de paquetes de .NET.

MVC — Model-View-Controller: Modelo-Vista-Controlador.

SPA — Single Page Application.

DOM — Document Object Model: árbol del documento.

UX — User Experience: experiencia de usuario.

UTF-8 — Codificación de caracteres Unicode de la web.

BOM — Byte Order Mark: marca invisible que rompe la codificación.

OWASP — Open Web Application Security Project.

XSS — Cross-Site Scripting: inyección de scripts.

CSRF — Cross-Site Request Forgery.

SSRF — Server-Side Request Forgery.

MFA — Multi-Factor Authentication.

HSTS — HTTP Strict Transport Security.

CSP — Content Security Policy.

JWT — JSON Web Token (gestión de sesión, semana 10).

PFC / EP1 — Proyecto Fin de Curso / Evaluación Parcial Primer Corte.

Síntesis de la Unidad II

Cruzamos del cliente al servidor: ahí viven la lógica de negocio, los datos y la seguridad.

Un mismo patrón, cuatro tecnologías: PERL/PHP, Spring Boot y ASP.NET Core resuelven request → proceso → response con distinta sintaxis.

Dos reglas de oro: validar TODA entrada en el servidor y escapar/parametrizar TODA salida y consulta.

La seguridad es transversal: OWASP Top 10 no se añade al final, se diseña desde la primera línea.

Siguiente: Unidad III — Gestión de estado, acceso a datos y arquitectura (semanas 10–13).